

作業實作指南

六人團隊版 | 從準備工具、訂規格，到實作、測試與驗收。

這份作業要做什麼

用 Python 做校園選課與行程規劃系統：管理課程與學生、檢查衝堂、產生課表、安排活動、搜尋最佳課表，並考慮移動時間與突發事件。

先看兩個階段

期中 F1～F6：資料管理 → 選課檢查 → 課表 → 活動與場地 → 課程篩選 → 儲存與測試。
期末 F7～F11：最佳課表 → 搜尋比較 → 最短路徑 → 事件重規劃 → AI 品質改善。

怎麼使用這份指南

先看第 2～3 頁，準備工具與資料規格，再依 F1～F11 順序開發。每頁都列出需要的東西、實作步驟與測試重點；最後整理紀錄、影片與驗收。

老師要求與建議做法

老師要求：Python、Git、核心函式可獨立測試、不用全域變數傳遞系統資料；命令列即可。
本指南建議：JSON、分模組、pytest；網頁可額外加做。資料欄位、時間規則及評分範例需經全組確認，不能直接當作老師指定。

資料來源

依使用者提供的「1151計算機程式設計團體作業.pdf」整理。此指南是開發參考，不取代團隊的需求分析、實作與驗證。

01 | 準備工具與專案

第 5 週先做好，後面才能讓六個人的程式接起來。

需要的東西

Python 3、Git、程式編輯器（例如 VS Code）、共用 Git Repository、會議與工作紀錄資料夾。

建議用 pytest 執行測試；JSON 可用 Python 內建 json。

常用指令：`python -m venv .venv`；啟用後 `python -m pip install pytest`。

執行入口：`python main.py`；執行測試：`python -m pytest`。

操作順序

1. 全組約定相同 Python 版本，建立專案與虛擬環境。
2. 建立 Git Repository；選一人維護整合版本。
3. 先寫 README：安裝方式、如何執行、如何跑測試。
4. 建立 modules、tests、data、docs 等資料夾。
5. 每人依功能開分支；測試通過、經別人檢視後再合併。

建議檔案

main.py：命令列入口；各功能放獨立模組。

tests/：自動化測試；data/：測試與儲存資料。

docs/：需求、會議、Review、缺陷、AI 與 TDD 紀錄。

requirements.txt：額外套件；README.md：執行說明。

完成檢查

另一位成員能從新下載的專案，依 README 執行最小程式與一個測試。不要依賴只有某台電腦才有的檔案路徑。

02 | 先統一資料與規則

沒有先約定格式，衝堂、活動、搜尋與儲存很容易各做各的。

核心資料

課程：代號、名稱、學分、必選修、教師、上限、教室、多個時段。

學生：學號、姓名、系級、已選課程、學分上限。

教室：代號、容量、設備、建築物（後三項供後續功能）。

活動：代號、名稱、日期、起迄、人數、類型、需求、場地、報名者。

另需選課紀錄、建築物與路線、限制與偏好。

時間表示範例

課程時段可用 (星期, 節次)，例如 (2, "3")、(2, "4")。星期是 1~7；節次順序固定為 1~9、a、b、c。

另建節次起迄表。活動日期轉星期後，再以實際時間與課程比較；團隊需訂學期日期範圍。建議時間區間採 [開始, 結束)，相接不算重疊。

第一次會議要決定

1. 空堂怎麼算？跨日的空間是否分開？
2. 選課紀錄以哪份資料為準？避免重複存兩份後不同步。
3. 已有人選課時，修改與刪除要如何處理？
4. 偏好是加分，禁止時段才是硬限制。
5. 找不到教室、無可行課表、資料損壞時，回傳什麼錯誤？

函式介面也先訂好

每個函式寫明：輸入、輸出、是否更新資料、錯誤與測試。例：check_enrollment(data, student_id, course_id) 回傳可否選課與問題清單；檢查本身先不要更新資料。

03 | F1 管理基本資料

目標：課程、學生及教室資料可以新增、修改、刪除與查詢。

開始前需要

全組確認的資料格式、欄位驗證規則、識別代號規則。驗收至少準備 20 門課程、15 名學生及 5 間教室。

實作步驟

1. 先做資料驗證：必填欄位、代號唯一、學分為正整數、人數上限至少 1。
2. 做新增與查詢，再做修改；修改前先驗證完整結果。
3. 做刪除：有人選修的課不能直接刪，須先退選或執行取消程序。
4. 做課程篩選查詢：代號、教師、星期、必選修、剩餘名額。
5. 準備固定測試資料，讓其他模組可以直接使用。

測試怎麼設計

正常：新增一門 3 學分課並查到它。

邊界：人數上限 1、星期 7、節次 c。

錯誤：重複代號、學分 0 或小數、節次 d、刪除已有人選的課。

修改失敗後，原資料應保持不變。

要留下的成果

F1 程式、資料模型、函式介面、測試資料與結果、非原開發者 Review。網頁加做：資料表與新增／修改表單。

04 | F2 選課與衝突檢查

目標：判斷能不能加選，並一次說明所有問題。

開始前需要

F1 資料、學生已選課程、活動與教室占用資料，以及全組共用的時段重疊函式。

實作步驟

1. 先確認學生與課程代號存在；無效輸入回傳明確錯誤。
2. 檢查與已選課程、已參加活動是否重疊。
3. 檢查教室占用、加選後學分、課程名額，另訂重複選課規則。
4. 把所有問題收集成清單；時間衝突按星期及節次排序。
5. 只有全部通過才更新選課；失敗不改任何選課紀錄。

例子

學生原有 22 學分，新課 3 學分，上限 24；新課也與微積分衝堂且已額滿。
結果應同時列出「衝堂、超過學分上限、額滿」，不能只顯示第一個問題。

驗證與成果

測無衝突、剛好學分上限、最後一個名額、同時多種問題、失敗後資料不變。
成果：檢查與加退選函式、排序規則、測試與 Review。網頁可把衝突時段標紅。

05 | F3 個人課表與空間

目標：看得懂一週安排，也能查連續空間時段。

開始前需要

已選課程、已參加活動、統一節次順序，以及空堂／跨日查詢的定義。課表為星期一至日、12 個節次。

實作步驟

1. 建立空的 7×12 課表矩陣，訂清楚列與欄代表什麼。
2. 把每門課的所有時段填入，再加入活動占用。
3. 提供依星期、節次、課程代號查詢。
4. 在指定範圍內掃描空格，找足夠長的連續空間；跨日按規格處理。
5. 統計每日上課節數、每週總學分及空堂；學分按課程計一次。

測試怎麼設計

同一門課每週多次出現；一天多門課；連續三節空間；跨日範圍；活動占用後空間縮短。
注意：一門 3 學分課出現在三個格子，不代表 9 學分。

要留下的成果

課表產生／查詢／空間／統計函式、輸出範例、測試。
網頁建議：課程與活動用不同顏色，空白格表示空間；遇到不合法的重疊資料，不能靜默覆蓋。

06 | F4 活動與教室配置

目標：活動能管理與報名，場地符合容量、設備和時間。

開始前需要

活動格式、日期與節次對照、教室容量與設備、課程／活動占用資料。與 F2 共用衝突檢查。

實作步驟

1. 做活動新增、修改、取消及查詢，驗證開始早於結束。
2. 依容量與設備篩教室，再查該日期時間是否被占用。
3. 選合適教室並登記；安排失敗時列出未滿足條件。
4. 做報名檢查：人數不能超過活動上限或教室容量。
5. 取消或修改活動時，同步釋放／重查占用，維持資料一致。

例子

30 人讀書會需要投影機。R101 只能坐 20 人、R102 已有課、R103 無投影機。
應列出每個不符合的原因；不能只顯示「安排失敗」。若多間都符合，團隊可訂選擇最小足夠容量等規則。

驗證與成果

至少 10 項活動；測多人報名、剛好容量、超額、場地衝突、取消釋放教室、修改後衝突。
成果：活動／報名／配置函式、共用時間檢查、測試。網頁：活動表與場地安排。

07 | F5 篩選 + F6 儲存測試

目標：先縮小候選課程，並讓關閉後的資料可以恢復。

F5 需要與做法

需要：希望修的課、必修、最低／最高學分、禁止時段、偏好星期。

1. 先驗證限制本身，例如最低學分不能大於最高。
 2. 篩候選課：禁止時段、額滿等；偏好依團隊規格處理。
 3. 指出明顯矛盾或最低學分不足；不能僅憑總學分夠就保證有可行課表。
- 至少 5 種情境，含矛盾、額滿及最低學分無法滿足。

F6 需要與做法

建議 JSON：方便存多時段與清單。儲存課程、學生、教室、活動、選課紀錄。

1. 定義檔案結構與版本，實作儲存及載入。
2. 逐筆驗證，壞資料依規則跳過並記錄，不讓整個程式異常終止。
3. 整份檔案損壞也要有錯誤回報；載入後檢查關聯資料。
4. 重開程式確認資料與儲存前一致；建議避免失敗時覆蓋有效舊檔。

期中測試門檻

至少 30 個自動化測試：正常至少 10、邊界至少 10、錯誤與衝突至少 10。測試須有預期結果與斷言，不是只印出資料。

pytest 可執行整套測試；人工截圖不能取代自動化結果。

期中完成檢查

F1～F6 能在同一套資料運作；v1.0、測試報告、AI 紀錄、個人報告 I、影片 I。

08 | F7 搜尋最佳課表

目標：找出所有可行課表，再依偏好分數排序。

開始前需要

必修、候選選修、學分上下限、禁止時段、偏好、既有安排的處理規則，以及 F2 的限制檢查。
第 10 週起先寫測試再實作。

實作步驟

1. 先寫小資料測試，人工列出預期可行組合，確認測試會失敗。
2. 建立候選組合；每個方案都查衝堂、必修、學分、名額及禁止時段。
3. 留下所有可行方案；定義順序無關的表示，方便去重與比對。
4. 計算偏好分數、排序；同分如何排序也寫進規格。
5. 重構並重跑舊測試；找不到方案時回傳明確原因。

評分範例：高分較好

分數 = 偏好課數 × 5 - 到校天數 × 3 - 空堂數 × 2 - 早八堂數 × 4。

此為建議，需自訂並解釋權重。提高到校天數的扣分，可能偏好更集中上課的方案。硬限制必須通過，不能只扣分。

測試與成果

測全部必修、學分剛好上下限、衝堂、禁止時段、額滿、無解、同分、權重改變排序。

成果：F7 規格、搜尋／評分、TDD、v1.1。網頁：比較課表與分數。

09 | F8 比較搜尋演算法

目標：證明回溯加速時沒有漏解，並用數據比較。

開始前需要

F7 可行性檢查；8、12、16 門候選課資料；一致的實驗環境、計時方式與節點定義。

實作步驟

1. 窮舉：每門候選課選或不選，產生所有組合再檢查。
2. 回溯：逐步加入課，遇到不能修復的違規就提早剪枝。
3. 對同一輸入比對完整可行解集合，不能只比解的數量。
4. 記錄搜尋節點、執行時間、可行解數，跑三種候選規模。
5. 若窮舉超時，設合理上限並記錄未完成；未完成不能宣稱解集合一致。

容易犯錯的剪枝

目前學分低於最低，不代表該分支無解，後面還可能加課。只有「剩下可加學分也不足」等可證明的情況才剪枝。

已超過最高學分則可剪枝，因為課程學分是正整數。

報告要有什麼

每組的資料、限制、節點定義、時間與解數、集合比對結果；說明為何某組剪枝較有效。

成果：程式、TDD、效能報告；網頁可畫時間與節點比較圖。

10 | F9 校園最短路徑

目標：兩堂課不只要時間不重疊，也要來得及走到。

開始前需要

至少 10 個建築物、15 條路線；教室對應建築物；節次起迄與課間空檔。訂清楚路線單向／雙向及時間單位。

實作步驟

1. 建立圖資料：鄰接表記錄每個建築物可到哪裡。
2. 成本相同用 BFS；步行時間不同用 Dijkstra（權重非負）。
3. 先寫小圖測試，手算最短路徑與時間，再實作。
4. 整理同一天相鄰課程，算前堂結束到後堂開始的分鐘數。
5. 最短移動時間大於空檔，標為不可行並接進 F7 搜尋。

例子與測試

A 棟到 B 棟需 15 分鐘，兩堂之間只有 10 分鐘：不可行。

測同一棟、剛好來得及、多條路、沒有路徑、無效地點。無路可達也要明確回報，不能假設移動時間是 0。

要留下的成果

圖資料、演算法說明、路徑與時間函式、TDD、F7～F9 整合 v1.2；第 13 週回歸後 v1.3。網頁可先顯示路徑清單，不必做複雜地圖。

11 | F10 事件後重新規劃

目標：發生變動後保留有效安排，找至少一項替代方案。

開始前需要

現有課表、資料更新功能、F7 搜尋、F9 移動限制；先訂「保留安排」與方案優先順序。

四種必做事件

課程額滿而需改選；教師改時間造成衝突；教室停用需重排；新增必參加活動造成衝突。

先訂清楚額滿情境：已成功選課通常不應因後來額滿被退選；可模擬待加選課已無名額或其他明確情況。

實作步驟

1. 先為四種事件寫失敗測試，保留原始安排快照。
2. 更新事件資料，重新檢查所有必要條件。
3. 標出仍有效與失效安排；呼叫既有搜尋／場地配置。
4. 建議先選更動最少，再比偏好分數；不可鎖死有效課程而漏掉唯一替代解。
5. 有解回傳前後差異；無解列原因，依規格決定資料提交方式。

測試與成果

測四種事件、有解／無解、盡量保留原課表、名額與移動時間仍符合、錯誤不造成半更新。

成果：F10、TDD、v1.4；網頁：事件操作與前後課表比較。

12 | F11 AI 協作與品質改善

目標：用測試判斷 AI 建議，不把 AI 回答當作驗證。

需要的東西

至少 3 個核心模組，每個至少 2 輪 AI 協作；原始程式版本、Prompt、回覆、修改理由、測試與結果。各成員整理自己的紀錄。

實作步驟

1. 先自行描述問題與預期行為，保存詢問前的版本。
2. 第一輪請 AI 分析結構、潛在錯誤或測試缺口。
3. 逐項用測試或推理檢查；採納或拒絕都記錄理由。
4. 第二輪提供實際測試結果或反例，請 AI 修正或補充。
5. 驗證改動並跑回歸，報告有效與錯誤／不適用的建議。

紀錄範例

問題：回溯可能漏解。

AI 建議：學分不足就剪枝。

驗證：後續還有課可加；建立反例，比對窮舉結果。

判斷：拒絕這項建議，改為確認剩餘學分仍無法達最低才剪枝。

這是示例；報告必須用實際發生的對話與測試。

要交的成果

AI 協作紀錄、品質改善報告、AI／人工測試比較、整合問題清單。第 15 週每人至少一項正確建議與一項錯誤／不適當建議的驗證。

13 | 六人合作與每週紀錄

每個人都要有可獨立測試的核心模組，也要理解別人的工作。

主責分工

A：整合、F6 儲存；B：F1；C：F2、F5；D：F3、F4；E：F7、F8；F：F9、F10。F11 全組參與、一人彙整。

E、F 的期末功能較晚開始，前期仍須協助需求、資料、測試與 Review。

每週團隊要留下

實體會議與照片；可執行版本；功能狀態表；需求／設計修改；自動化測試；Git；非原開發者 Review；缺陷；約一頁 Iteration Review。第 10 週起保留完整 TDD。

每週個人要留下

工作內容與日期、Git 提交、問題與解法；一個重要函式的 Input／Process／Output／邊界錯誤；至少一項他人程式 Review；AI 建議與驗證；第 10 週起實際參與的 TDD。

可以共用的紀錄欄位

缺陷：編號／版本／重現步驟／預期／實際／主責／修正與測試。

Review：模組／作者／檢視者／問題／修改／驗證。

AI：問題／Prompt／建議／判斷理由／版本／測試。

TDD：需求／先寫測試的版本／失敗結果／實作／重構／回歸。

14 | 展示、影片與最後驗收

影片呈現真實解題歷程；最後每人都要能現場解釋與修改。

影片 I：第 8 週，6～8 分鐘

從 F1～F6 選一個實際問題。包含情境、分解、詢問前解法、至少兩輪 Prompt、不能直接採用的建議與驗證、修改前後、團隊反思。原則上各成員參與說明。

影片 II：第 17 週，8～10 分鐘

從 F7～F10 選複雜問題。比較至少兩種解法；展示需求 → 測試 → Red → Green → Refactor → 回歸；呈現 AI 協作與錯誤、實驗數據、與期中四面向比較。每人約 20～30 秒反思。

兩支影片都要附

影片檔或連結，以及片中 Prompt、程式版本、測試與實驗數據。內容必須可追溯，不能只口頭說做過。建議每週保存素材，不要期末才找。

第 17～18 週完成檢查

1. 全系統 F1～F11 重跑測試、修正整合問題，產出 v2.0。
2. 整理程式、README、需求設計、測試、效能、AI、Git 與個人紀錄。
3. 換一台或乾淨環境，照說明啟動並載入測試資料。
4. 練習正常流程、失敗流程、事件與替代方案的展示。
5. 互相解釋模組，練習小功能修改與修錯；驗收後交最終修正版。